JAVASCRIPT INTERVIEW QUESTIONS

Q1. What is JavaScript? What is the role of JavaScript engine?

Ans. JavaScript is a lightweight, high-level, interpreted programming language. It is mainly used to add interactivity and dynamic behavior to web pages. It can run both on the client-side (in browsers) and server-side (using Node.js).

The JavaScript Engine is a special program that executes JavaScript code. Popular engines are V8 (Chrome & Node.js), SpiderMonkey (Firefox), and JavaScriptCore (Safari).

Role of JavaScript Engine:

- It first performs tokenization (breaking code into small tokens like keywords, operators, variables).
- Then it does parsing and creates an Abstract Syntax Tree (AST) — a tree-like structure representing the code.
- After that, it interprets or compiles the AST into machine code using Just-In-Time (JIT) compilation for better speed.
- It also handles memory management through Garbage Collection.

In simple words, the JavaScript Engine converts human-readable JS code into something the computer can understand and run efficiently.

Q2. What are client and server in web developemnt?

Ans. **Client** refers to the user's device and web browser (like Chrome, Firefox, or Safari). It is where the user interacts with the website. The client sends requests to the server and receives responses.

Server is a powerful computer (or program) that runs 24/7 and stores the website's data, files, and business logic. It processes requests coming from clients and sends back the required data or web pages.

In web development:

- **Client-side** means code that runs in the browser (mainly HTML, CSS, and JavaScript).
- **Server-side** means code that runs on the server (Node.js, Python, Java, PHP, etc.).

JavaScript can run on both client (browser) and server (Node.js). The client handles user interface and interactivity, while the server handles data processing, security, and database operations.

Q3. What is "undefined" and "null" in JavaScript?

Ans. **undefined** is a primitive data type in JavaScript. It is automatically assigned by the JavaScript engine to a variable that has been declared but not yet initialized with a value.
null is also a primitive value, but it is intentionally assigned by the developer to indicate that the variable has no value or is empty.
Key difference: undefined means "value is not assigned", while null means "value is absent" (deliberately set).
Type of undefined is "undefined", and type of null is "object" (this is a famous bug in JavaScript).

You can explicitly assign undefined to a variable like this:

```
let i = undefined;
```

It will work perfectly and the variable i will hold the value undefined. However, in real-world coding, it is not recommended. Good practice is to use null when you want to intentionally say "no value".

Q4. What will be the output of undefined == null & undefined === null? Why?

Ans.

- undefined == null → **true**
- undefined === null → **false**

Reason:
The loose equality (==) operator does **type coercion**. JavaScript converts both undefined and null into the same value for comparison, so it returns true.

But the strict equality (===) operator checks both **value and data type** without coercion. Since undefined and null have different types, it returns false.

Q5. What is hoisting in javascript ?

Ans. Hoisting is a default behavior of JavaScript where the JavaScript engine moves all variable and function declarations to the top of their scope before the code is executed.

This means you can use variables and functions before they are declared in the code, but with some limitations.

How Hoisting Works:

- **var** declarations are hoisted and initialized with undefined. Example: You can access console.log(a) before var a = 10; (it prints undefined).
- **let** and **const** are hoisted but **not initialized**. They stay in the Temporal Dead Zone (TDZ). Accessing them before declaration gives a ReferenceError.
- **Function declarations** are fully hoisted (both declaration and definition), so you can call them before they appear in the code.
- **Function expressions** and arrow functions are not hoisted like declarations.

  Hoisting allows us to use function declarations before declaring them, but we must be very careful with variables (var, let, const) to avoid unexpected undefined or ReferenceError.

Q5. Explain Temporal Dead Zone.

Ans. Temporal Dead Zone (TDZ) is the time period between the start of the scope and the line where let or const variable is declared and initialized.

During this TDZ, if you try to access the variable, JavaScript throws a **ReferenceError**: "Cannot access 'variable' before initialization".

Deep Explanation:

- Even though let and const are hoisted (their declaration is moved to the top of the block), they are **not initialized** with any value (unlike var which gets undefined).
- JavaScript keeps these variables in a special "dead zone" until the execution reaches the actual declaration line.
- Once the declaration line is executed, the variable is initialized and comes out of TDZ. After that, you can safely use it.
- TDZ exists only for let and const. It does **not** exist for var.

It was added in ES6 to catch bugs early and to make code more predictable. It forces developers to declare variables before using them.

Q6. What is the scope of a variable ?

Ans. **Scope** of a variable is the region of the code where that variable can be accessed and used. It defines the visibility and lifetime of a variable in JavaScript.

JavaScript has mainly 3 types of scopes:

1. **Global Scope**
    - Variables declared outside any function or block are in global scope.
    - They can be accessed from anywhere in the code (inside functions, blocks, or even other scripts).
    - Example: let name = "NxtWave";

- Global variables stay in memory until the program ends (not recommended to overuse).

2. **Function Scope (Local Scope)**
   - Variables declared inside a function using var, let, or const are in function scope.
   - They can only be accessed inside that function.
   - Each function creates its own scope.
   - Example: Variables declared with var inside a function are not accessible outside.
3. **Block Scope**
   - Introduced in ES6 with let and const.
   - Variables declared inside { } (curly braces) like if, for, while, or any block are block-scoped.
   - They cannot be accessed outside that block.
   - var does **not** have block scope — it ignores { } and stays function-scoped.

Important Point:

Var → Function Scoped

let & const → Block Scoped

This is why modern JavaScript prefers let and const over var.

Q7. What is the scope of a variable ?

Ans. No, it is **not mandatory** to terminate every line with a semicolon (;) in JavaScript because of **Automatic Semicolon Insertion (ASI)**.

The JavaScript engine automatically inserts semicolons at the end of statements in most cases when it detects a line break.

However, it is a **best practice** to always use semicolons for these reasons:

- It makes your code more readable and predictable.

- It prevents unexpected bugs caused by ASI rules (especially when using return, throw, break, or when starting a new line with (, [, /, or +).
- Many companies and coding standards (like Airbnb, Google) strictly follow semicolon style.

Always use semicolons at the end of statements. It is safer, cleaner, and shows professionalism in interviews and real projects. In Strict Mode ('use strict') does not disable Automatic Semicolon Insertion (ASI), but it makes ASI behavior more strict and predictable.

Q8. what are the data types in JavaScript?

Ans. JavaScript has two categories of data types: **Primitive** and **Non-Primitive** (Reference) types.

Primitive Data Types (Stored by value):

- **Number** → Represents both integers and floating-point numbers (e.g., 42, 3.14)
- **String** → Represents textual data ("hello", 'world')
- **Boolean** → true or false
- **Undefined** → Default value of uninitialized variables
- **Null** → Intentional absence of any value
- **Symbol** → Unique and immutable primitive value (used for object keys)
- **BigInt** → For very large integers (e.g., 123456789012345678901n)

Non-Primitive Data Type:

In JavaScript, **Non-Primitive** data types are also called **Reference Types**. The main Non-Primitive data type is **Object**.

All the following are considered **Objects** internally by the JavaScript engine:

- **Objects** → Simple key-value pairs {name: "Chandu"}
- **Arrays** → Ordered list of values [1, 2, 3]

- **Functions** → Reusable blocks of code function(){}
- **Dates, RegExp, Maps, Sets**, etc.

Primitive types are immutable and passed by value, while Objects are mutable and passed by reference.

Q9. What are the functions in JavaScript?

Ans. In JavaScript, a function is a reusable block of code that performs a specific task. Functions are first-class citizens in JavaScript. Functions can be assigned to variables, passed as arguments, and returned from other functions.

In JavaScript, there are mainly **three types** of functions:

1. **Function Declaration** This is the traditional way to create a function. It is **hoisted**, meaning you can call the function before it is declared in the code.
2. **Function Expression** A function is assigned to a variable. It is not hoisted, so you cannot call it before declaration.
3. **Arrow Function** A shorter and cleaner syntax introduced in ES6. It does not have its own this binding (it takes this from the surrounding scope). Arrow functions are not hoisted.
4. **IIFE** stands for **Immediately Invoked Function Expression**. It is a JavaScript function that is defined and **executed immediately** after it is created. The function is wrapped in parentheses () so the JS engine treats it as an expression, not a declaration. It runs only **once** when the script loads. Variables declared inside IIFE are not accessible outside (private scope).

Q10. Explain difference between the var, let and const keywords.

Ans. In JavaScript, var, let, and const are used to declare variables, but they behave differently.

Here are the main differences:

1. **Scope**:
    - var → Function-scoped (or global if declared outside function)
    - let and const → Block-scoped (limited to {} curly braces)
2. **Hoisting**:
    - var → Hoisted and initialized with undefined
    - let and const → Hoisted but **not initialized** (Temporal Dead Zone)
3. **Re-declaration and Re-assignment**:
    - var → Can be re-declared and re-assigned
    - let → Can be re-assigned but **cannot** be re-declared in the same scope
    - const → Cannot be re-assigned or re-declared (must be initialized at declaration)

Best Practice: Always use const by default. Use let only when you need to re-assign the value. Avoid var in modern JavaScript.

Q11. Difference between " == " and " === " operators.

Ans. The == operator is called loose equality, while === is strict equality in JavaScript.

- == compares values after type coercion, meaning it automatically converts one value's type to match the other.
  For example, "5" == 5 returns true because the string is converted into a number.

- In contrast, === first compares the data types, and if they are not the same, it immediately returns false.
  If the types are equal, then it compares the values.
  So "5" === 5 returns false, but 5 === 5 returns true.

In practice, === is preferred because it avoids unexpected behavior and makes code more reliable.

Q12. Is javascript a statically typed or a dynamically typed language?

Ans. JavaScript is a **Dynamically Typed** language.

Explanation:

- In dynamically typed languages, the data type of a variable is checked **at runtime**, not at compile time.
- You don't need to declare the data type while declaring a variable. The same variable can hold different types of values at different times.

Comparison:

- **Statically Typed** (like Java, TypeScript): Type is checked at compile time. You must declare type (e.g., int x = 10;).
- **Dynamically Typed** (like JavaScript, Python): Type is decided at runtime and can change.

Q13. Can we declare a variable in JavaScript without using a keyword?

Ans. Yes, we can declare a variable without `var`, `let`, or `const` by simply assigning a value, like `x = 10`.
In this case, JavaScript automatically creates a **global variable** (property of the global object).
However, this only works in **non-strict mode**; in strict mode (`"use strict"`), it throws a `ReferenceError`.
Such variables are called **implicitly declared variables**.
This practice is not recommended because it can lead to **global scope pollution and bugs**.

Q14. What is JSON?

Ans. JSON stands for **JavaScript Object Notation**, which is a lightweight data format used for storing and exchanging data.
It is written in **key-value pairs**, similar to JavaScript objects.
JSON is **language-independent**, meaning it can be used with many programming languages like Java, Python, etc.
Data in JSON is usually in **string format**, so it needs to be converted using `JSON.parse()` (string → object) and `JSON.stringify()` (object → string).
It is widely used in **APIs and web applications** to send and receive data between client and server.
JSON is easy to read, write, and understand, which makes it very popular in modern development.

Q15. What is the difference between unary, binary, and ternary operators?

Ans. In JavaScript, operators are classified based on the number of operands they work with:

- Unary Operators → Operate on one operand.

  Example: typeof, ! (negation), ++ (increment), -- (decrement), - (unary minus).

- Binary Operators → Operate on two operands.

  Most common operators fall here.

  Example: +, -, *, /, ==, ===, &&, ||, =.

- Ternary Operator → Operates on three operands.

  It is the only ternary operator in JavaScript, also called conditional operator.

  Syntax: condition ? valueIfTrue : valueIfFalse

Q16. What is Short-Circuit Evaluation in Javascript

Ans. Short-circuit evaluation in JavaScript means that logical operators (`&&`, `||`) **stop evaluating as soon as the result is determined**.

Here is a breakdown of how it works with the two primary logical operators:

1. Logical OR (`||`)

The `||` operator returns the first **truthy** value it encounters. If the first operand is truthy, it "short-circuits" and doesn't even look at the second operand.

2. Logical AND (`&&`)

The `&&` operator returns the first **falsy** value it encounters. If the first operand is falsy, it stops immediately because the whole expression can no longer be true.

Q17. What are the types of conditional statements in JS?

Ans. In JavaScript, conditional statements are used to make decisions based on different conditions. The main types are.

- **`if` statement**, which executes code when a condition is true.
- **`if...else` statement** is used to choose between two blocks of code.
- **`if...else if...else`** is used when we have multiple conditions to check.
- The **`switch` statement** is used to handle multiple cases based on a single value.
- Additionally, the **ternary operator (`condition ? expr1 : expr2`)** is a shorthand for simple `if...else`.

These help control the flow of execution in a program based on logic.

Q18. Explain Closures in JavaScript.

Ans. A **closure** in JavaScript is a function that remembers variables from its outer (lexical) scope even after the outer function has finished execution. This happens because functions in JavaScript form a closure with the scope in which they were created.

Closures are mainly used to **preserve state** and create **private variables**. Even when the outer function is removed from the call stack, the inner function still has access to those variables.

Q19. What is function currying? Explain practical usage of function currying.

Ans. Function currying is a technique where a function with multiple arguments is transformed into a sequence of functions, each taking one argument at a time. Instead of calling `f(a, b, c)`, we call it as `f(a)(b)(c)`. This helps in breaking down complex functions into smaller reusable functions.

In JavaScript, currying is often implemented using closures, where each returned function remembers the previous arguments. It improves code reusability and readability, especially in functional programming patterns.

A practical use case is creating reusable utility functions. For example, a `multiply(a)(b)` function allows you to create a preset function like `double = multiply(2)`, and reuse it for different values.

Currying is also useful in event handling and partial application, where some arguments are fixed in advance. Libraries like React and Redux use currying concepts for cleaner state management and middleware handling.

Overall, currying helps write modular, composable, and maintainable code.

Q20. What is NaN? When do you get NaN as output?

Ans. `NaN` stands for **"Not a Number"**, and it is a special value in JavaScript that represents an invalid or undefined numeric result. Its type is still `"number"`, which is a common interview trick.

You get `NaN` when a mathematical operation fails to produce a valid number. For example, trying to convert a non-numeric string using `Number("abc")` or doing `"hello" * 2` will result in `NaN`.

It also occurs in operations like `0 / 0`, `Math.sqrt(-1)` (in real numbers), or undefined numeric calculations.

A key point is that `NaN` is **not equal to itself**, so `NaN === NaN` returns `false`. To check it properly, we use `isNaN()`.

Q21. Explain the must know points of arrow function.

Ans. Arrow functions are a concise way to write functions in JavaScript using `=>` syntax. They reduce boilerplate code, especially for short functions and callbacks.

The most important feature is that arrow functions **do not have their own** `this`. Instead, they inherit `this` from the surrounding (lexical) scope, which is very useful in callbacks, event handlers, and classes.

Arrow functions cannot be used as constructors, meaning you cannot call them with `new`, and they don't have a `prototype` property.

They support implicit return, so for single expressions we can skip `return` and curly braces, making code cleaner.

Another key point is they are always anonymous, so they are usually assigned to variables or used inline.

Q22. What are generator functions? Explain the syntax.

Ans. Generator functions are special functions in JavaScript that can **pause and resume execution**, instead of running completely at once. They are used to control iteration flow and produce values one by one.

The syntax uses `function*` (note the *), and inside the function we use the `yield` keyword to pause execution and return a value.

```
function* myGenerator() {
  yield 1;
  yield 2;
  yield 3;
}
```

Calling a generator function does not execute it immediately. Instead, it returns a **generator object (iterator)**.

```
const gen = myGenerator();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Each call to `next()` resumes execution from the last `yield`.

Q23. How does Memory Management and Garbage Collection work in JS?

Ans. JavaScript uses **automatic memory management**, meaning the engine allocates and frees memory without developer intervention. Garbage Collection (GC) is the process of removing unused memory to avoid leaks.

The most common algorithm used is **Mark-and-Sweep**. In this, the GC starts from "root" objects (like global variables, current function scope) and marks all reachable objects. Any object not reachable is considered garbage and gets deleted.

Memory lifecycle has three steps: allocation (when variables/objects are created), usage (read/write operations), and release (when GC removes unused data).

An important concept is **reachability**. If an object is no longer referenced anywhere in the program, it becomes eligible for garbage collection.

Closures and global variables can prevent GC if references are still held, which may lead to memory leaks if not handled carefully.

Q24. How do you handle errors in JavaScript code?

Ans. Error handling in JavaScript is mainly done using `try...catch...finally` blocks to catch runtime errors and prevent the application from crashing.

```
try {
  let result = riskyFunction();
} catch (error) {
  console.log(error.message);
} finally {
  console.log("Execution completed");
}
```

The `try` block contains code that may throw an error, `catch` handles the error, and `finally` runs always (cleanup tasks).

JavaScript provides a `throw` keyword to create custom errors.

For asynchronous code, errors are handled using `.catch()` in Promises or `try...catch` with `async/await`.

Common error types include `ReferenceError`, `TypeError`, and `SyntaxError`.

Q25. Explain various ways to declare a string variable.

Ans. In JavaScript, strings can be declared in multiple ways depending on use case and flexibility.

The most common way is using **single quotes (`' '`)** or **double quotes (`" "`)**. Both work the same, and choice depends on consistency or avoiding escape characters.

Another modern way is using **template literals (backticks `` ` ` ``)**. These support **string interpolation** and multi-line strings.

You can also create strings using the **String constructor**, but it is rarely used in practice. `let str1 = new String("Hello");`

Q26. What is a template literal? What is the advantage of using template string?

Ans. A **template literal** is a modern way to create strings in JavaScript using backticks (`` ` ` ``) instead of single or double quotes. It allows embedding expressions directly inside the string using `${}` syntax.

One major advantage is **string interpolation**, where variables and expressions can be inserted easily without using `+` concatenation. This makes code cleaner and more readable.

Template literals also support **multi-line strings** without using `\n`, which improves formatting. They can also handle **expressions and function calls** inside `${}`, making them powerful for dynamic content.

Q27. Explain the indexOf() and lastIndexOf() method with syntax.

Ans. `indexOf()` and `lastIndexOf()` are string methods used to find the position of a substring inside a string. Both return the index (starting from 0), or `-1` if the value is not found.

indexOf() searches from **left to right** and returns the first occurrence.

lastIndexOf() searches from **right to left** and returns the last occurrence.

Q28. Explain substring() and slice().

Ans. `substring()` and `slice()` are used to extract a part of a string based on index positions. Both return a new string without modifying the original string.

`substring(start, end)` extracts characters from `start` to `end` (end not included).

`slice(start, end)` works similarly but is more flexible.

Key differences:

- `slice()` supports **negative indexes**, so it can extract from the end.
- `substring()` does **not support negative values** (it treats them as 0).
- If `start > end`, `substring()` swaps values, but `slice()` returns an empty string.

Q29. Explain immutability in strings.

Ans. In JavaScript, **strings are immutable**, which means once a string is created, its value **cannot be changed**. Any operation on a string returns a **new string** instead of modifying the original.

```
let str = "hello";
str[0] = "H";
console.log(str); // still "hello"
```

Even though it looks like we are modifying the string, JavaScript does not allow direct changes to individual characters.

If we perform operations like `toUpperCase()` or `replace()`, a new string is created:

This means the original string remains unchanged, and the result is stored separately.

Q30. Explain different ways of creating date/time object.

Ans. In JavaScript, date and time are handled using the `Date` **object**. There are multiple ways to create it, each used in different scenarios.

1. Current date and time

Creates a Date object with the system's current date and time. Used in real-time applications like timestamps and logging.

```
let now = new Date();
console.log(now);
```

2. Using milliseconds (timestamp)

Pass the number of milliseconds since **Jan 1, 1970 (UTC)**. Useful when working with stored timestamps from databases.

```
let date = new Date(0);
console.log(date); // 1970-01-01
```

3. Using date string

You can pass a date in string format. Common in APIs where dates come as strings.

```
let date = new Date("2024-06-15");
console.log(date);
```

4. Using individual components

Pass year, month, day, hour, minute, second. Note: Month is **0-based** (0 = January, 5 = June).

```
let date = new Date(2024, 5, 15, 10, 30, 0);
```

5. Using `Date.now()`

Returns current timestamp in milliseconds. Used for performance tracking and calculations.

```
let timestamp = Date.now();
```

Q31. What is an array in JS? In how many ways can we create arrays?

Ans. An **array in JavaScript** is a special object used to store multiple values in a single variable. These values can be of any type (numbers, strings, objects, etc.), and each element is accessed using an index starting from 0. Arrays are widely used to manage lists of data.

There are **multiple ways to create arrays**:

1. Array literal (most common and recommended)

You directly use square brackets.

Example: let arr = [1, 2, 3, 4]

This is simple, fast, and used in most real-world code.

2. Using Array constructor

You use the `Array` keyword.

Example: `let arr = new Array(1, 2, 3)`

This works like a literal, but is less preferred because it can be confusing.

3. Using Array constructor with length

If you pass a single number, it creates an empty array with that length.

Example: `let arr = new Array(5)`

This creates an array with 5 empty slots, not values.

4. Using Array.of()

Creates an array from given arguments.

Example: `let arr = Array.of(5)`

This avoids confusion of constructor and creates [5].

5. Using Array.from()

Creates an array from iterable or array-like objects.

Example: `let arr = Array.from("hello") → ['h','e','l','l','o']`

Used when converting strings, NodeLists, etc.

6. Using spread operator

Creates a new array by spreading values from another iterable.

Example: `let arr = [..."abc"] → ['a','b','c']`

Useful for copying or merging arrays.

Q32. Explain array methods to add and remove elements in array ?

Ans. JavaScript provides several built-in array methods to **add and remove elements**. These methods either modify the original array (mutable) or return a new array.

1. push() – add at end

Adds one or more elements to the end of the array.

Example: let arr = [1, 2]; arr.push(3) → [1, 2, 3]
Used when you want to append data.

2. pop() – remove from end
Removes the last element and returns it.
Example: let arr = [1, 2, 3]; arr.pop() → [1, 2]
Commonly used like stack (LIFO).

3. unshift() – add at beginning
Adds elements to the start of the array.
Example: let arr = [2, 3]; arr.unshift(1) → [1, 2, 3]
Used when order matters from the front.

4. shift() – remove from beginning
Removes the first element and returns it.
Example: let arr = [1, 2, 3]; arr.shift() → [2, 3]
Works like a queue (FIFO).

Q33. Explain splice() method in JS

Ans. `splice()` is a powerful array method used to **add, remove, or replace elements at any position** in an array. It **modifies the original array** (mutable).

Syntax:
array.splice(startIndex, deleteCount, item1, item2, ...)

- `startIndex` → position to start changes
- `deleteCount` → number of elements to remove
- `items` → elements to add (optional)

1. Removing elements
Example: let arr = [1, 2, 3, 4]; arr.splice(1, 2) → removes [2, 3], result: [1, 4]

2. Adding elements
Example: let arr = [1, 4]; arr.splice(1, 0, 2, 3) → [1, 2, 3, 4]

3. Replacing elements
Example: let arr = [1, 2, 3]; arr.splice(1, 1, 5) → [1, 5, 3]

`splice()` method **returns the removed elements** as an array.

Q34. What is the difference between find() and filter() method?

Ans. `find()` and `filter()` are array methods used to search elements based on a condition, but they differ in output and usage.

find() returns the **first element** that satisfies the condition. If no element matches, it returns `undefined`.
Example: let arr = [1, 2, 3, 4]; arr.find(x => x > 2) → 3
Used when you only need one matching value.

filter() returns **all elements** that satisfy the condition as a new array. If no match, it returns an empty array `[]`.
Example: let arr = [1, 2, 3, 4]; arr.filter(x => x > 2) → [3, 4]
Used when multiple results are needed.

Key differences:

- `find()` → single value, stops after first match
- `filter()` → multiple values, checks all elements
- `find()` → returns element or undefined
- `filter()` → always returns array

In real-world usage, use `find()` for searching one item (like user by ID) and `filter()` for getting lists (like all active users).

Q35. What is a template literal? What is the advantage of using template string?

Ans. In JavaScript, **iterator methods** are mainly used with arrays to loop through elements and perform operations. These are also called **iterative methods**. They internally use iteration logic and accept callback functions that **apply a callback function** to each item. They are widely used for data processing.

1. forEach()

Executes a function for each element, does not return a value.

Syntax: arr.forEach((element, index, array) => { })

Arguments: element → current value, index → position, array → original array

2. map()

Returns a new array by transforming each element.

Syntax: arr.map((element, index, array) => { })

Arguments: element → current value, index → position, array → original array

3. filter()

Returns a new array with elements that satisfy a condition.

Syntax: arr.filter((element, index, array) => { })

Arguments: element → current value, index → position, array → original array

4. find()

Returns the first element that matches the condition.

Syntax: arr.find((element, index, array) => { })

Arguments: element → current value, index → position, array → original array

5. some()

Checks if at least one element satisfies the condition (returns true/false).

Syntax: arr.some((element, index, array) => { })
Arguments: element → current value, index → position, array → original array

6. every()
Checks if all elements satisfy the condition (returns true/false).
Syntax: arr.every((element, index, array) => { })
Arguments: element → current value, index → position, array → original array

7. reduce()
Reduces array to a single value.
Syntax: arr.reduce((accumulator, element, index, array) => { }, initialValue)
Arguments: accumulator → accumulated result, element → current value, index → position, array → original array

---

Q36. How can you sort an array ?

Ans. In JavaScript, arrays can be sorted using the `sort()` **method**. It sorts the elements of an array **in place** (modifies the original array).

Basic syntax:
array.sort()

By default, `sort()` converts elements to strings and sorts them in **lexicographical (dictionary) order**.
Example: let arr = [10, 2, 5]; arr.sort() → [10, 2, 5] (wrong for numbers)

To sort numbers correctly, we use a **compare function**:
Syntax: array.sort((a, b) => a - b)

- If result < 0 → a comes before b
- If result > 0 → b comes before a
- If result = 0 → no change

Example (ascending):
let arr = [10, 2, 5];

arr.sort((a, b) => a - b) → [2, 5, 10]

Example (descending):
let arr = [10, 2, 5];

arr.sort((a, b) => b - a) → [10, 5, 2]

Q37. Explain Array destructuring ?

Ans. **Array destructuring** is a feature in JavaScript that allows you to **extract values from an array and assign them to variables in a single line**. It improves readability and reduces manual indexing.

Basic syntax:
let [a, b] = [10, 20]
Here, `a = 10` and `b = 20`

Skipping values:
let [a, , b] = [1, 2, 3] → a = 1, b = 3
Used when you don't need certain elements.

Default values:
let [a = 5, b = 10] = [1] → a = 1, b = 10
Helpful when array may have missing values.

Using rest operator:
let [a, ...rest] = [1, 2, 3, 4] → rest = [2, 3, 4]
Used to collect remaining elements.

Swapping variables:

let a = 1, b = 2; [a, b] = [b, a]

Useful without using a temporary variable.

Q38. What is object in JS ? In how many way can we create objects?

Ans. An **object in JavaScript** is a collection of **key-value pairs**, where keys are called properties and values can be any data type (number, string, function, array, etc.). Objects are used to represent real-world entities and store structured data.

There are **multiple ways to create objects**:

1. Object literal (most common and recommended)

You can define an object using {}.

Example: let obj = { name: "Chandu", age: 22 }

Used in almost all real-world scenarios because it is simple and readable.

2. Using `new Object()` constructor

Creates an empty object and then you add properties.

Example: let obj = new Object(); obj.name = "Chandu"

Less preferred compared to literal syntax.

3. Using constructor function

You create a function and use new to create objects.

Example: function Person(name) { this.name = name }

let p1 = new Person("Chandu")

Used when creating multiple similar objects.

4. Using `Object.create()`

Creates a new object with a specified prototype.

Example: let obj = Object.create({ greet: "hello" })

Used when working with prototypes and inheritance.

5. Using ES6 class

Modern way to create objects using class syntax.

Example: class Person { constructor(name) { this.name = name } }

let p1 = new Person("Chandu")

Used in large applications for better structure.

6. Using factory function

A function that returns an object.

Example: function createUser(name) { return { name } }

Used for flexible object creation without `new`.

Q39. What is the "this" keyword? Explain.

Ans. The "**`this`**" **keyword** in JavaScript refers to the **object that is currently executing the function**. Its value is **not fixed** and depends on how the function is called (this is a key interview point).

1. Global scope

In the global context, `this` refers to the global object (in browser → `window`).

Example: console.log(this)

2. Inside a regular function

In normal functions, `this` depends on how the function is called.

If called normally → `this` is global (or `undefined` in strict mode).

3. Inside an object method

When a function is called as a method of an object, `this` refers to that object.

Example: let obj = { name: "Chandu", show() { console.log(this.name) } }

Here, `this` → obj

4. Inside arrow function

Arrow functions do **not have their own `this`**. They take `this` from the

surrounding scope (lexical this).
This is useful in callbacks to avoid losing context.

5. Constructor function / class
When using `new`, `this` refers to the newly created object.
Example: function Person(name) { this.name = name }

6. Using call(), apply(), bind()
We can manually control `this`.
Example: func.call(obj) → sets `this` to obj

Q40. Explain call(), apply(), bind()

Ans. `call()`, `apply()`, and `bind()` are JavaScript methods used to **control the value of `this` inside a function**. They are very important when you want to reuse a function with different objects.

call()
The `call()` method invokes (executes) a function **immediately** and allows you to pass the `this` value explicitly. Arguments are passed **one by one (comma separated)**.
Use case: when you want to borrow a function from another object and execute it instantly with a specific context.

apply()
The `apply()` method is similar to `call()`, it also **executes the function immediately** and sets `this`. The only difference is that arguments are passed as an **array (or array-like object)**.
Use case: useful when arguments are already available in array form, like dynamic data or calculations.

bind()
The `bind()` method does **not execute the function immediately**. Instead, it

returns a **new function** with `this` permanently bound to the given object. You can call that returned function later.
Use case: commonly used in event handlers, callbacks, and situations where you want to preserve `this` for future execution.

Key difference:

- `call()` → executes immediately, arguments individually
- `apply()` → executes immediately, arguments as array
- `bind()` → returns new function, executes later

Q41. Explain Destructuring in Objects.

Ans. **Object destructuring** is a feature in JavaScript that allows you to **extract properties from an object and assign them to variables in a clean and concise way**. It avoids repeatedly accessing properties using dot notation.

Basic syntax
You match variable names with object keys.
Example: let { name, age } = { name: "Chandu", age: 22 }
Here, variables `name` and `age` get values directly.

Renaming variables
You can assign properties to different variable names.
Example: let { name: userName } = obj
Used when variable name should differ from key.

Default values
If a property is missing, you can set a default value.
Example: let { city = "Hyderabad" } = obj
Prevents undefined values.

Nested destructuring

You can extract values from nested objects.
Example: let { address: { city } } = obj
Used in complex data like API responses.

Rest operator

You can collect remaining properties.
Example: let { name, ...rest } = obj
Useful when separating required and remaining data.

Q42. What is DOM or DOM API? What do you mean by DOM Manipulation?

Ans. The **DOM (Document Object Model)** is a programming interface provided by the browser that represents an HTML document as a **tree structure of objects**. Each element (like `<div>`, `<p>`, `<h1>`) becomes a node, which JavaScript can access and control.

The **DOM API** is a set of built-in methods and properties (like `getElementById`, `querySelector`) that allow JavaScript to interact with and modify the DOM.

DOM Manipulation means **changing the structure, content, or style of the webpage dynamically using JavaScript**. This includes adding, removing, or updating elements.

Examples of manipulation (in text form):

- Changing content → element.innerText = "Hello"
- Changing style → element.style.color = "red"
- Adding element → parent.appendChild(newElement)
- Removing element → element.remove()

Q43. What is the difference between window & document object ?

Ans. The **`window` object** is the **global object** in the browser. It represents the entire browser window and contains all global variables, functions, and browser-related APIs like `alert()`, `setTimeout()`, `location`, and `history`.

The **`document` object** is a **property of the window object** (`window.document`). It represents the **HTML page (DOM)** loaded inside the browser and allows you to access and manipulate elements.

Key differences:

- **Scope:** `window` → whole browser environment, `document` → only the webpage content
- **Purpose:** `window` handles browser-level operations, `document` handles page structure (DOM)
- **Access:** `window` is global, `document` is accessed through `window.document`
- **Usage:** `window` for alerts, navigation, timers; `document` for selecting and modifying HTML elements

Example understanding:

- window → controls browser (tabs, URL, timers)
- document → controls HTML elements inside the page

Q44. What are the events in JS. Explain.

Ans. **Events in JavaScript** are actions or occurrences that happen in the browser, which JavaScript can detect and respond to. These actions can be triggered by the user (like clicking, typing) or by the browser (like page load).

An event works with an **event handler (function)** that runs when the event occurs. This is the core of making web pages interactive.

Common types of events:

1. Mouse events
Triggered by mouse actions like click, double click, hover.
Examples: `click`, `dblclick`, `mouseover`
Use case: button clicks, opening menus.

2. Keyboard events
Triggered when user presses keys.
Examples: `keydown`, `keyup`
Use case: form input validation, shortcuts.

3. Form events
Related to form elements.
Examples: `submit`, `change`, `focus`, `blur`
Use case: validating inputs, handling form submission.

4. Window events
Triggered by browser actions.
Examples: `load`, `resize`, `scroll`
Use case: loading data when page opens, responsive design.

5. Input events
Triggered when user types or modifies input fields.
Example: `input`
Use case: live search, auto suggestions.

Key concept:
Events follow a flow called **event propagation** (capturing → target → bubbling), which controls how events travel in the DOM.

Q45. Explain event propagation (capturing → target → bubbling) clearly.

Ans. **Event propagation** is the process that defines **how an event travels through the DOM tree** when it is triggered. It happens in three phases: **capturing → target → bubbling**.

1. Capturing phase (trickling phase)

The event starts from the **root (document/window)** and travels **downwards** to the target element.

In this phase, parent elements get the chance to handle the event **before the actual target**.

By default, this phase is not used unless explicitly enabled.

2. Target phase

This is the point where the event **reaches the actual element** that triggered it.

Here, the event handler attached directly to the target element gets executed.

3. Bubbling phase

After reaching the target, the event travels **back upwards** from the target to the root.

Parent elements can handle the event in this phase.

By default, most events in JavaScript follow **bubbling**.

Q46. Explain syntax of addEventListener() method.

Ans. **addEventListener()** is used to attach an event handler to an element without overwriting existing handlers. It gives better control compared to inline or traditional event methods.

Syntax:

```
element.addEventListener(eventType, callbackFunction, options)
```

- `eventType` → type of event (like "click", "keydown")
- `callbackFunction` → function that runs when event occurs
- `options` (optional) → controls behavior like capturing (`true`) or bubbling (`false` by default)

Key points:

- You can attach **multiple listeners** to the same event
- Supports **capturing phase** if enabled
- More flexible and widely used in modern JS

Q47. What is an event object?

Ans. An **event object** is an object automatically created when an event occurs. It contains **detailed information about the event**.

It is passed as a parameter to the event handler function.

Example (text): function handleClick(event)

Common properties:

- `event.target` → element that triggered the event
- `event.type` → type of event (click, keydown)
- `event.clientX / clientY` → mouse position
- `event.key` → key pressed in keyboard event

Use case:
Event object helps to **understand what happened and where**, making it essential for handling user interactions.

Q48. Explain the difference between setTimeout() and setInterval()

Ans. `setTimeout()` and `setInterval()` are **browser timer functions** used to handle **asynchronous behavior** in JavaScript. They allow code to run after a delay without blocking the main execution thread.

setTimeout()
This method executes a function **only once after a specified delay (in milliseconds)**.
Syntax: setTimeout(callback, delay)
Use case: showing a message after some time, delaying execution, or simulating async tasks.

setInterval()
This method executes a function **repeatedly at a fixed time interval**.
Syntax: setInterval(callback, interval)
Use case: updating UI continuously, timers, clocks, or polling APIs.

Key differences:

- `setTimeout()` → runs once after delay
- `setInterval()` → runs continuously at intervals
- `setTimeout()` → used for one-time tasks
- `setInterval()` → used for repeated tasks

Stopping timers:

- clearTimeout(id) → stops setTimeout
- clearInterval(id) → stops setInterval

Q49. What is a callback function ?

Ans. A **callback function** is a function that is **passed as an argument to another function** and is executed later, either after some operation completes or when a specific event occurs.

It is a core concept in JavaScript used to handle **asynchronous behavior** and event-driven programming.

In simple terms, it means: *"Call this function back when you are done with your work."*

Callbacks are commonly used in:

- Event handling (like click, input events)
- Asynchronous operations (like timers, API calls)
- Array methods (map, filter, forEach)

They help JavaScript run non-blocking code, meaning the program does not wait for one task to finish before starting another.

A callback can be **synchronous** (executed immediately) or **asynchronous** (executed later after some delay or event).

Q50. What is the meaning of callback hell or pyramid of doom?

Ans. **Callback hell (also called Pyramid of Doom)** is a situation in JavaScript where **multiple nested callback functions are used inside each other**, making the code **very difficult to read, maintain, and debug**.

It usually happens when we perform **multiple asynchronous operations one after another**, where each step depends on the previous one. Instead of writing clean linear code, callbacks become deeply nested.

The structure looks like a pyramid shape, where indentation keeps increasing, hence the name **pyramid of doom**.

Problems caused by callback hell:

- Hard to read and understand
- Difficult to debug and maintain
- Poor code scalability
- Error handling becomes complex

Why it happens:
JavaScript handles async tasks using callbacks, and when many dependent async tasks are chained, nesting increases.

How it is solved in modern JavaScript:

- Promises (better chaining using `.then()`)
- Async/Await (clean, synchronous-like code structure)

Q51. What is the typeof operator?

Ans. typeof returns a string representing the operand's type.
Common results:

- typeof 1 → "number"
- typeof 'a' → "string"
- typeof true → "boolean"
- typeof undefined → "undefined"

- typeof Symbol() → "symbol"
- typeof 10n → "bigint"
- typeof function(){} → "function" (functions are callable objects)
- typeof {} → "object"

  Quirk: typeof null returns "object"

Q52. What is strict mode?

Ans. Strict mode ("use strict") is a restricted variant of JavaScript that makes errors throw where non-strict mode might silently fail, prevents certain unsafe actions, and disallows some deprecated features. It helps catch bugs and improves performance opportunities for JS engines.

Key effects:

- Prevents accidental global variable creation.
- this in functions defaults to undefined (not global object).
- Disallows duplicate property names or parameter names.
- Disallows with statement.
- Throws on assignment to non-writable properties, etc.

Q53. What is call stack?

Ans. The The **call stack** in JavaScript is a data structure that keeps track of **function execution order**. It works on the **LIFO principle (Last In, First Out)**, meaning the last function added is executed first and removed first.

When a function is called, it is **pushed onto the call stack**. When the function finishes execution, it is **popped off the stack**.

The call stack handles all **synchronous code execution** in JavaScript. Only one function runs at a time because JavaScript is single-threaded.

If one function calls another, the new function is added on top of the stack, and control returns to the previous function after it completes.

If too many nested calls happen, it can lead to a **stack overflow error** (maximum call stack exceeded).

Q54. What is an Asynchronous process?

Ans. An **asynchronous process** in JavaScript means that a task **does not block the execution of other code**. Instead of waiting for a task to finish, JavaScript continues executing the next lines and handles the result later. This is important for tasks that take time, like API calls, timers, or file operations.

For example, operations like **network requests, timers (`setTimeout`) and events** run asynchronously, so the program stays responsive.

Now, **Is JavaScript asynchronous?**
JavaScript is actually **single-threaded and synchronous by nature**, meaning it executes one line at a time. But it behaves asynchronously using features provided by the environment (like browser APIs or Node.js).

This async behavior is handled using:

- **Callbacks**
- **Promises**
- **async/await**

Behind the scenes, concepts like the **Event Loop, Web APIs, and Callback Queue** help manage asynchronous execution.

Q55. What is a Promise in JS? Explain

Ans. A **Promise in JavaScript** is an object used to handle **asynchronous operations**. It represents a value that may be available **now, later, or never**. It helps avoid callback hell and makes async code cleaner and more manageable.

A Promise has **three states**:

- **Pending** → initial state, operation is still running
- **Fulfilled (resolved)** → operation completed successfully
- **Rejected** → operation failed

Promises are created using the `Promise` constructor, where you pass a function with two parameters: `resolve` and `reject`. These are used to indicate success or failure of the operation.

To handle results, we use:

- `.then()` → runs when promise is fulfilled
- `.catch()` → runs when promise is rejected
- `.finally()` → runs always (cleanup tasks)

Promises allow **chaining**, so multiple async steps can be written in a clean sequence instead of nested callbacks.

In real-world usage, Promises are used for **API calls, database operations, file handling**, etc.

Q56. What is chaining in the promise? Explain the syntax.

Ans. **Promise chaining** means executing multiple asynchronous operations **one after another in a sequence**, where the output of one step is passed to the next. It helps avoid deeply nested callbacks and keeps code clean and readable.

In chaining, each `.then()` returns a **new Promise**, so you can attach another `.then()` to it. This creates a chain of operations.

Syntax (text form):

```
promise
.then(result1 => { return nextValue })
.then(result2 => { return anotherValue })
.catch(error => { handleError })
```

Each `.then()` receives the result from the previous step. If you return a value, it is passed to the next `.then()`. If you return a Promise, the next `.then()` waits for it to resolve.

Error handling is done using `.catch()`, which will handle errors from any step in the chain.

Q57. Explain functionality of async/await ?

Ans. **`async/await`** is a modern way to handle **asynchronous operations** in JavaScript, built on top of Promises. It allows you to write async code that looks **synchronous and easy to read**.

An **`async`** **function** always returns a Promise. Even if you return a normal value, JavaScript automatically wraps it inside a resolved Promise.

The **`await`** **keyword** is used inside an async function to **pause execution until a Promise is resolved or rejected**. It makes the code wait for the result without blocking the entire program.

Syntax (text form):

```
async function example() {
let result = await somePromise
return result
}
```

If the Promise is resolved, `await` gives the result. If it is rejected, it throws an error, which can be handled using **try...catch**.

This approach removes complex `.then()` chains and makes the flow look like normal step-by-step execution.

Q58. Explain the fetch API functionality.

Ans. The **Fetch API** is a modern JavaScript feature used to make **HTTP requests** (like GET, POST) to servers. It is promise-based and is commonly used to **fetch data from APIs** in web applications.

The main function is `fetch()`, which takes a URL and optional configuration (method, headers, body). It returns a **Promise** that resolves to a **Response object**.

Syntax (text form):

```
fetch(url, options)
.then(response => response.json())
.then(data => handleData(data))
.catch(error => handleError(error))
```

The response is not directly usable, so we call methods like `response.json()` to convert it into usable data. This step is also asynchronous.

Fetch supports different request types like **GET, POST, PUT, DELETE**, and allows sending headers and body data (like JSON).

Q59. What are object prototypes?

Ans. **Object prototypes** in JavaScript are a mechanism by which **objects can inherit properties and methods from other objects**. Every JavaScript object has an internal link called **`[[Prototype]]`**, which points to another object.

This forms a **prototype chain**, where if a property is not found in the current object, JavaScript looks up in its prototype, and continues until it reaches `null`.

For example, arrays and functions also have prototypes, so they can use built-in methods like `push()` or `toString()` through this chain.

You can access or set prototypes using `__proto__`, `Object.getPrototypeOf()`, or `Object.setPrototypeOf()`.

Functions in JavaScript have a special property called **`prototype`**, which is used when creating objects with `new`. Those objects inherit from that prototype.

Q60. What is memoization?

Ans. **Memoization** is an optimization technique used to **improve performance by storing (caching) the results of expensive function calls**. If the same input occurs again, the function returns the cached result instead of recalculating it.

In simple terms, it means: *"Don't repeat work if you already know the answer."*

When a function runs, memoization saves the input and its output in a cache (usually an object or map). Next time, before executing, it checks if the result already exists.

This is very useful for functions that are **called multiple times with the same inputs**, especially in heavy computations like recursion (e.g., Fibonacci).

How it works (concept):

- Check if input exists in cache
- If yes → return cached result
- If no → compute, store in cache, return result

Benefits:

- Reduces repeated calculations
- Improves performance
- Saves time in complex operations

Q61. What is the constructor function in javascript? What is the use of a constructor function?

Ans. A **constructor function** in JavaScript is a special type of function used to **create and initialize objects**. It acts like a blueprint for creating multiple objects with the same structure and behavior. By convention, constructor function names start with a **capital letter**.

When you call a constructor function using the `new` **keyword**, JavaScript does four things:

- Creates a new empty object
- Sets `this` to that new object
- Links the object to the function's prototype
- Returns the object automatically

Inside the constructor, properties are assigned using `this`, so each object gets its own copy of data.

Use of constructor function:

- To create multiple similar objects easily
- To avoid repeating code

- To define shared methods using prototype
- To implement object-oriented patterns

It is widely used in large applications where many objects of the same type are needed, like users, products, or vehicles.

Q62. What do you mean by BOM?

Ans. **BOM (Browser Object Model)** refers to the set of objects provided by the browser that allow JavaScript to **interact with the browser itself**, not just the web page content.

It represents the **browser window and its components**, and the main object in BOM is the `window` **object**, which is the global object in the browser.

Through BOM, JavaScript can perform actions like:

- Navigating URLs (`location`)
- Accessing browser history (`history`)
- Showing alerts (`alert`, `confirm`)
- Working with timers (`setTimeout`, `setInterval`)
- Getting screen information (`screen`)

Unlike the DOM, which deals with HTML elements, BOM deals with **browser-level features**.

Each browser may implement BOM slightly differently, so it is not fully standardized like the DOM.

Q63. Explain Local storage and session storage and cookies

Ans. **Local Storage, Session Storage, and Cookies** are used in JavaScript to **store data on the client side (browser)**, but they differ in lifetime, capacity, and usage.

1. Local Storage

It stores data **permanently (no expiration)** until manually cleared by the user or code.

Data remains even after closing and reopening the browser.

It stores data in **key-value pairs** and has a larger size limit (~5–10 MB).

Use case: saving user preferences, themes, or app data.

2. Session Storage

It stores data only for the **current browser session**.

Data is cleared automatically when the **tab or browser is closed**.

It also uses key-value pairs but is limited to a single tab.

Use case: temporary data like form inputs or session-based state.

3. Cookies

Cookies store small pieces of data (around **4 KB**) and are **sent to the server with every HTTP request**.

They can have an **expiration time** or be session-based.

Used for authentication, tracking, and storing user sessions.

Key differences:

- Lifetime: localStorage (long-term), sessionStorage (session), cookies (configurable)
- Size: local/session storage > cookies
- Server communication: cookies are sent to server, others are not

---

Q64. What are classes in javascript?

Ans. **Classes in JavaScript** are a modern way (introduced in ES6) to create **objects and implement object-oriented programming (OOP)** concepts. A class acts like a **blueprint** for creating multiple objects with the same properties and methods.

Inside a class, we define a special method called `constructor`, which is used to initialize object properties when a new object is created. Methods defined in a class are shared by all instances through the prototype.

Classes support important OOP features like **encapsulation, inheritance, and abstraction**. Using `extends`, one class can inherit properties and methods from another class.

Although classes look similar to other languages like Java, internally JavaScript still uses **prototype-based inheritance**. Classes are just a cleaner syntax over prototypes.

They also support features like **static methods** (called on class, not objects) and **getters/setters** for controlled access to properties.

Use cases:

- Creating structured and reusable code
- Building large-scale applications
- Managing complex data models

Q65. Explain pass-by-reference and pass-by-value in JS

Ans. In JavaScript, **pass-by-value** and **pass-by-reference (behavior)** describe how values are passed to functions.

Pass-by-value means a **copy of the value is passed**, so changes inside the function do not affect the original variable.
This happens with **primitive types** like number, string, boolean, null, undefined, and symbol.

Example (text): let a = 10 → function gets a copy, modifying it does not change the original a.

Pass-by-reference (behavior) is seen with **objects and arrays**. Here, the value passed is actually a **reference (address) to the object**, so changes inside the function can affect the original object.
Example (text): let obj = {x: 1} → modifying obj.x inside function changes original object.

Important clarification: JavaScript is technically **pass-by-value**, but for objects, the value itself is a **reference**, so it behaves like pass-by-reference.

Key differences:

- Primitives → copy, no effect on original
- Objects → reference shared, changes reflect outside

Use cases:

- Use primitives when you want safe copies
- Be careful with objects to avoid unintended mutations

Q66. Explain clearly about event loop in JS

Ans. The **event loop** in JavaScript is a mechanism that allows JavaScript to handle **asynchronous operations** even though it is **single-threaded**. It coordinates between the **call stack**, **Web APIs (browser/Node APIs)**, and **task queues** to execute code efficiently.

1. Call Stack
This is where JavaScript executes functions one by one (LIFO order).
Synchronous code runs here first.

2. Web APIs

When async functions like `setTimeout`, `fetch`, or DOM events are called, they are handled by browser APIs outside the call stack.

3. Callback Queue (Task Queue)

Once an async task is completed, its callback is placed in the queue, waiting to be executed.

4. Event Loop Working

The event loop continuously checks:

- If the call stack is empty → it takes a callback from the queue
- Pushes it into the call stack for execution

5. Microtask Queue (important)

Promises (`.then`, `catch`, `finally`) go into a **microtask queue**, which has **higher priority** than the normal callback queue. So microtasks run before macrotasks like `setTimeout`.

Flow summary:

- Run synchronous code → call stack
- Async tasks go to Web APIs
- Completed tasks move to queues
- Event loop pushes them back to call stack when empty

Q67. What are optional chaining (?.) and nullish coalescing (??)?

Ans. **Optional chaining (`?.`)** and **nullish coalescing (`??`)** are modern JavaScript operators that help handle **missing or undefined values safely**.

Optional chaining (`?.`)

It is used to **safely access nested properties** of an object without causing errors. If any part of the chain is `null` or `undefined`, it **stops execution and returns**

`undefined` instead of throwing an error.
Syntax (text): obj?.property?.subProperty
Use case: accessing API data where some fields may not exist.

Nullish coalescing (`??`)
It is used to provide a **default value only when the value is `null` or `undefined`**.
Syntax (text): value ?? defaultValue
Unlike `||`, it does not treat `0`, `false`, or empty string as false.

Key differences:

- `?.` → safe property access
- `??` → default value assignment

Example concept:
If a value is missing → `??` gives default
If a property may not exist → `?.` avoids error

Q68. What is heap memory?

Ans. **Heap memory** in JavaScript is the region of memory used to **store objects, arrays, and functions dynamically**. Unlike the call stack, which is ordered, the heap is a **large, unstructured memory area** where data is stored and accessed using references.

When you create an object or array, it is allocated in the heap, and a **reference (memory address)** to that data is stored in the variable.

This is why when you assign one object to another variable, both variables point to the **same heap memory**, not a copy.

Heap memory is managed automatically by JavaScript using **garbage collection**, which removes objects that are no longer reachable.

Key points:

- Stores complex data (objects, arrays, functions)
- Works with references, not direct values
- No fixed order like stack

Q69. What is a higher-order function?

Ans. A **higher-order function** in JavaScript is a function that **either takes another function as an argument or returns a function as its result**. This is possible because functions in JavaScript are treated as **first-class citizens** (they can be stored in variables, passed, and returned).

Such functions help in writing **more reusable and modular code**, especially in functional programming patterns.

Common examples are array methods like `map`, `filter`, and `reduce`, where a function is passed as a callback to process each element.

A higher-order function can also return another function, which is useful for creating **customized or partially applied functions**.

Use cases:

- Data transformation (map, filter)
- Event handling
- Creating reusable logic

Benefits:

- Cleaner and more readable code
- Reduces repetition
- Improves flexibility

Q70. What is the rest parameter?

Ans. The **rest parameter** in JavaScript is used to **collect multiple arguments into a single array**. It allows a function to accept **any number of arguments** without knowing the exact count in advance.

It is written using the `...` **(three dots)** before the parameter name.
Syntax (text): function func(...args)

Inside the function, `args` becomes an **array** containing all remaining arguments passed to the function.

It is useful when the number of inputs is dynamic, like summing values or handling variable data.

Important rules:

- Rest parameter must be the **last parameter** in the function
- Only **one rest parameter** is allowed
- It replaces the older `arguments` object with a cleaner approach

Q71. What does flat() do?

Ans. The `flat()` method in JavaScript is used to **flatten a nested array into a single-level array**. It removes inner arrays and combines their elements into one array.

By default, `flat()` flattens only **one level deep**. That means it will only remove one layer of nesting.

Syntax (text): array.flat(depth)

- `depth` is optional and defines how many levels to flatten
- If you pass `2`, it flattens two levels

- If you pass `Infinity`, it flattens all nested levels completely

Example concept:
[1, [2, 3], [4, [5]]] → flat() → [1, 2, 3, 4, [5]]

Important point: `flat()` **does not modify the original array**, it returns a **new flattened array**.

Use cases:

- Cleaning nested data
- Working with multi-dimensional arrays
- Simplifying data before processing

Q72. What is Object.keys() and Object.values() and Object.entries() ?

Ans. `Object.keys()`, `Object.values()`, and `Object.entries()` are built-in JavaScript methods used to **extract data from objects in array form**, making it easier to loop and process.

Object.keys()
It returns an array of all the **property names (keys)** of an object.
Syntax (text): Object.keys(obj)
Use case: when you want to iterate over keys or check available properties.

Object.values()
It returns an array of all the **values** of an object.
Syntax (text): Object.values(obj)
Use case: when you only care about the values (like summing numbers).

Object.entries()
It returns an array of **key-value pairs**, where each pair is an array `[key, value]`.

Syntax (text): Object.entries(obj)
Use case: useful for looping with both key and value together.

Key differences:

- keys() → returns only keys
- values() → returns only values
- entries() → returns both as pairs

Q73. What is Set?

Ans. A **Set** in JavaScript is a built-in object used to **store unique values**. It does not allow duplicate elements, so if you try to add the same value again, it will be ignored. Sets can store any type of value (numbers, strings, objects).

Creating a Set

You can create a set using the `Set` constructor.
Syntax (text):
let set = new Set()
You can also initialize it with values:
let set = new Set([1, 2, 3, 3]) → duplicates are automatically removed

Key features of Set:

- Stores only **unique values**
- Maintains insertion order
- Methods like `add()`, `delete()`, `has()`

Use cases:

- Removing duplicates from arrays
- Storing unique items
- Fast lookup operations

WeakSet

A **WeakSet** is similar to Set, but it has some important differences.

- It can store **only objects** (not primitive values)
- It holds **weak references**, meaning objects can be garbage collected if no other reference exists
- It is **not iterable** (you cannot loop over it)

Use case:

Used for storing object references without preventing memory cleanup, useful in memory-sensitive applications.

Key difference:

- Set → stores all types, strong references, iterable
- WeakSet → stores only objects, weak references, not iterable

---

Q74. What is freeze() and seal()?

Ans. `Object.freeze()` and `Object.seal()` are methods used to **control how an object can be modified** in JavaScript. They help make objects more secure and prevent unwanted changes.

Object.freeze()

This method makes an object **completely immutable**.

- You cannot add new properties
- You cannot delete existing properties
- You cannot modify existing values

In simple terms, the object becomes **read-only**.

Object.seal()
This method partially restricts changes.

- You cannot add or delete properties
- But you **can modify existing property values**

So, the structure is locked, but values can still change.

Key differences:

- `freeze()` → no changes at all (fully immutable)
- `seal()` → structure locked, values editable

Q75. What is cache and caching?

Ans. A **cache** is a storage layer used to **temporarily store data** so that it can be accessed faster in the future. Instead of recomputing or fetching data again, the system can reuse the stored result.

Caching is the process of **saving frequently used or expensive-to-fetch data into the cache** and retrieving it from there when needed. It improves performance and reduces unnecessary operations.

In JavaScript or web applications, caching can happen at different levels:

- **Browser cache** → stores static files like images, CSS, JS
- **Memory cache** → stores data in variables or objects
- **API cache** → stores server responses

How it works:

- First request → data is fetched or computed and stored in cache
- Next request → data is returned from cache (faster)

Benefits:

- Faster performance
- Reduced server load
- Better user experience

Q76. What is querySelector and querySelectorAll?

Ans. `querySelector()` and `querySelectorAll()` are **DOM API methods** used to **select HTML elements using CSS selectors**. They are more flexible and powerful than older methods like `getElementById`.

querySelector()
It returns the **first element** that matches the given CSS selector.
Syntax (text): document.querySelector("selector")
If no match is found, it returns `null`.
Use case: when you only need a single element.

querySelectorAll()
It returns **all matching elements** as a **NodeList (collection)**.
Syntax (text): document.querySelectorAll("selector")
If no elements match, it returns an empty NodeList.
Use case: when you need to work with multiple elements.

Key differences:

- `querySelector()` → returns first match
- `querySelectorAll()` → returns all matches
- `querySelectorAll()` → result can be looped using `forEach`

Example concept:

".class" → select by class

"#id" → select by id

"div p" → select nested elements

Q77. Difference between innerHTML and textContent?

Ans. `innerHTML` and `textContent` are DOM properties used to **get or set content inside an element**, but they behave differently.

innerHTML

It reads or writes the **HTML content** inside an element, including tags.

If you set it, the browser parses the string as HTML and renders elements.

Use case: dynamically inserting HTML structure (like adding `<b>`, `<div>`).

Risk: can cause **XSS security issues** if untrusted input is inserted.

textContent

It reads or writes only the **plain text** inside an element.

It does not interpret HTML tags; it treats them as normal text.

Use case: safely inserting user input or text data.

Key differences:

- `innerHTML` → works with HTML + text, parses tags
- `textContent` → works with plain text only
- `innerHTML` → slower (parsing involved)
- `textContent` → faster and safer

Example concept:

If value is "Hello"

- innerHTML → renders bold "Hello"
- textContent → shows "Hello" as text

Q78. What is a buffer?

Ans. A **buffer** is a temporary memory area used to **store data while it is being transferred or processed**. It acts as an intermediate space between two systems or operations that may work at different speeds.

In JavaScript (especially in Node.js), a buffer is used to handle **binary data** like files, streams, images, or network data. Since JavaScript normally works with strings and objects, buffers help manage raw data efficiently.

Why buffer is needed:
When data is coming from a source (like a file or network), it may not arrive all at once. A buffer **collects the data in chunks** until it is ready to be processed.

Key characteristics:

- Fixed-size memory allocation
- Stores raw binary data
- Faster processing for streams

Use cases:

- File reading/writing
- Streaming data (audio, video)
- Network communication

Q79. What is parseInt?

Ans. `parseInt()` is a built-in JavaScript function used to **convert a string into an integer (whole number)**. It reads the string from left to right and stops when it finds a non-numeric character.

Syntax (text):
parseInt(string, radix)

- `string` → the value to convert
- `radix` (optional) → the base of the number (e.g., 10 for decimal, 2 for binary)

How it works:

- Converts valid numeric part of string to integer
- Ignores characters after invalid input
- Returns `NaN` if conversion is not possible

Example concept:
"123abc" → 123
"abc123" → NaN

If radix is not provided, JavaScript usually assumes base 10, but it's good practice to always specify it.

Use cases:

- Converting user input to numbers
- Parsing data from strings
- Working with different number bases

Q80. Explain HTTP and HTTPS

Ans. **HTTP (HyperText Transfer Protocol)** and **HTTPS (HyperText Transfer Protocol Secure)** are protocols used for **communication between a client (browser) and a server** on the web. They define how data is requested and transferred.

HTTP

It is the basic protocol used to load web pages. Data is sent in **plain text**, which means it is not secure and can be intercepted by attackers.

Default port: **80**

Use case: simple or non-sensitive data transfer.

HTTPS

It is the secure version of HTTP. It uses **encryption (SSL/TLS)** to protect data during transmission. This ensures **confidentiality, integrity, and authentication**.

Default port: **443**

Use case: secure communication like login, payments, and personal data.

Key differences:

- HTTP → not secure, data in plain text
- HTTPS → secure, data encrypted
- HTTPS uses **SSL/TLS certificates** to verify identity

Why HTTPS is important:

- Prevents data theft (man-in-the-middle attacks)
- Builds user trust (lock icon in browser)
- Required for modern web features

Q81. Explain HTTP request and HTTP response.

Ans. An **HTTP request** and **HTTP response** are the two main parts of communication between a **client (browser)** and a **server**.

HTTP Request

An HTTP request is sent by the client to the server asking for some resource (like a webpage, API data).

Elements of an HTTP request:

- **Request Line** → contains method, URL, and HTTP version
  Example: GET /api/users HTTP/1.1
- **Method** → action to perform (GET, POST, PUT, DELETE)
- **Headers** → additional information (content type, authorization, etc.)
- **Body (optional)** → data sent to server (used in POST/PUT requests)

Use case: requesting data from server, submitting forms, calling APIs.

HTTP Response

An HTTP response is sent by the server back to the client with the requested data or status.

Elements of an HTTP response:

- **Status Line** → contains HTTP version, status code, and message
  Example: HTTP/1.1 200 OK
- **Status Code** → indicates result (200 success, 404 not found, 500 error)
- **Headers** → metadata about response (content type, cache info)
- **Body** → actual data returned (HTML, JSON, etc.)

Use case: sending webpage content, API responses, error messages.

Key Difference:

- Request → sent by client to ask for data
- Response → sent by server to provide data or result

Q82. What is structuredClone?

Ans. `structuredClone()` is a built-in JavaScript method used to create a **deep copy of a value**. It copies not just the top-level object, but also all **nested objects and data**, producing a completely independent clone.

Unlike shallow copy methods, `structuredClone()` ensures that **no references are shared**, so changes in the copied object do not affect the original.

It supports many data types like **objects, arrays, Map, Set, Date, RegExp, and even nested structures**. It can also handle circular references, which many older methods cannot.

Syntax (text):

structuredClone(value)

Key advantages:

- True deep copy (all levels)
- Handles complex and circular data
- Safer and more reliable than JSON-based copying

Limitations:

- Does not clone functions or DOM elements
- Not supported in very old browsers

Q83. Explain different types of loops in JS.

Ans. JavaScript provides several types of loops to **repeat a block of code**, each suited for different use cases.

1. for loop

Used when you know how many times you want to iterate. It has initialization,

condition, and increment.
Syntax (text): for (init; condition; update)
Use case: looping with index control.

2. while loop
Runs as long as the condition is true. The condition is checked before execution.
Syntax: while (condition)
Use case: when number of iterations is unknown.

3. do...while loop
Similar to while, but executes at least once before checking condition.
Syntax: do { } while (condition)
Use case: when code must run at least once.

4. for...of loop
Used to iterate over **iterable values** like arrays, strings, maps.
Syntax: for (let value of iterable)
Use case: directly accessing values.

5. for...in loop
Used to iterate over **object keys (properties)**.
Syntax: for (let key in object)
Use case: looping through object properties.

6. forEach() (array method)
Executes a function for each element in an array.
Syntax: array.forEach(callback)
Use case: simple iteration without break/continue.

Key differences:

- `for`, `while`, `do...while` → general loops
- `for...of` → values of iterable
- `for...in` → keys of object
- `forEach()` → array method with callback